B23 PyTorch foundations

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B23 PyTorch foundations 3
B23-L1 Tensors, shape, dtype and shared memory 3
B23-L2 Autograd, gradients from the chain rule 7
B23-L3 Gradient descent, learning rates and optimisers 12
B23-L4 Models as nn.Module, layers, activations and parameters 17
B23-L5 Loss functions, logits and target shapes 23
B23-L6 Dataset, DataLoader and preprocessing fitted on training rows 28
Track B best-practice checklist 36
Appendix: solutions and answer keys 37

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B23 PyTorch foundations

Roadmap: Months 1 to 3: PyTorch foundations. Tensors, autograd, gradient descent, nn.Module, loss functions and data loading, built up from NumPy.

B23-L1 Tensors, shape, dtype and shared memory

Concept

A **tensor** is PyTorch's array. It works like a NumPy array, and it also records gradients (lesson B23-L2) and runs on a GPU (lesson B16-L1). Examples use PyTorch 2.15. PyTorch does not run in this app's browser runtime, so the code exercises use NumPy, which follows the same shape and memory rules.

Every tensor has a **shape** and a **dtype**:

```
x = torch.zeros(4, 3)
x.shape                              # torch.Size([4, 3])
x.sum(dim=0).shape                   # torch.Size([3])
x.sum(dim=1, keepdim=True).shape     # torch.Size([4, 1])
x.unsqueeze(0).shape                 # torch.Size([1, 4, 3])
x.reshape(-1).shape                  # torch.Size([12])
```

dtype. PyTorch's default float is `float32`. NumPy's is `float64`, and `torch.tensor(df.to_numpy())` keeps `float64`. Layer weights are `float32`, so the first forward pass fails with "mat1 and mat2 must have the same dtype, but got Double and Float". Convert once, at the edge: `torch.as_tensor(X, dtype=torch.float32)`. Integers become `int64`.

Shared memory. `torch.from_numpy(a)` and slices such as `x[:64]` share memory with their source, as NumPy slices do. Change one and the other changes. `torch.tensor(a)` copies. An in-place operation, such as `batch -= batch.mean(dim=0)`, on a slice therefore changes the full dataset. Write `batch = batch - batch.mean(dim=0)` to get a new tensor.

Broadcasting. Shapes line up from the right, and a dimension of 1 stretches to match. `(3, 1) - (3,)` gives `(3, 3)`, not `(3,)`. This is the most common silent bug in training code: predictions of shape `(n, 1)` against targets of shape `(n,)` produce an `n` by `n` table of differences, and the mean of that table looks like a plausible loss. Flatten predictions with `reshape(-1)` or `squeeze(1)` before comparing, and assert shapes where data enters a function.

Why it matters

A model of spend per respondent stopped improving after a few epochs, at a loss that never went below the variance of the target. Predictions had shape (n, 1) and targets shape (n,). Every prediction was being compared with every target, so the best the model could do was predict the mean. One `squeeze(1)` fixed it, and the loss fell by two thirds.

Worked example: weak code

```
import torch

X = torch.tensor(df[FEATURES].to_numpy())
y = torch.tensor(df["spend"].to_numpy())
error = model(X) - y
```

1. **Line 3.** float64 data meets float32 weights: the forward pass fails.
2. **Line 5.** (n, 1) minus (n,) broadcasts to (n, n). No error, wrong numbers.

Worked example: strong code

```
import numpy as np
import torch


def to_tensors(X: np.ndarray, y: np.ndarray) -> tuple[torch.Tensor, torch.Tensor]:
    """Features and target as float32 tensors with checked shapes.

    Args:
        X: Feature matrix, one row per respondent.
        y: Target, one value per respondent.

    Returns:
        X as (n, k) and y as (n,), both float32.

    Raises:
        ValueError: If X is not 2-D or the row counts differ.
    """
    if X.ndim != 2 or len(X) != len(y):
        raise ValueError(f"expected X (n, k) and y (n,), got {X.shape} and {y.shape}")
    return torch.as_tensor(X, dtype=torch.float32), torch.as_tensor(y, dtype=torch.float32).reshape(-1)


def residuals(model: torch.nn.Module, X: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """Prediction minus target, one value per row."""
    preds = model(X).squeeze(1)
    assert preds.shape == y.shape, (preds.shape, y.shape)
```

```
27     return preds - y
```

1. **Line 19.** Check shapes where data enters.
2. **Line 20.** Convert to float32 once, at the edge, and make the target 1-D.
3. **Line 25.** (n, 1) becomes (n,), the same shape as the target.
4. **Line 26.** A wrong shape fails here, not as a strange loss three epochs later.

Convert to float32 at the edge, know which operations share memory, and make predictions and targets the same shape before you compare them.

Recap card

- **PyTorch and NumPy default floats** PyTorch float32, NumPy float64. Convert with torch.as_tensor(X, dtype=torch.float32) at the edge.
- **What shares memory** torch.from_numpy, view and slices share memory. torch.tensor copies. In-place ops on a shared tensor change the source.
- **(n, 1) against (n,)** Broadcasts to (n, n) without an error. Flatten with squeeze(1) or reshape(-1) and assert shapes.
- **Shape after sum(dim=1, keepdim=True) on (4, 3)** (4, 1). Without keepdim: (4,).

Practice

Exercise 1. Multiple choice (level 1)

The first forward pass fails with "mat1 and mat2 must have the same dtype, but got Double and Float". The features came from `torch.tensor(df[FEATURES].to_numpy())`. What do you change?

- a) Build the tensor with `torch.as_tensor(X, dtype=torch.float32)`
- b) Convert the model to float64 with `model.double()`
- c) Round the features to whole numbers
- d) Move the model to the GPU

Why?

- r1) NumPy data is float64 and layer weights are float32. Converting the data once, at the edge, keeps the model in its default dtype.
- r2) float32 tensors always give more accurate results.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
1 import torch
2
3 x = torch.zeros(4, 3)
```

```
print(x.sum(dim=0).shape, x.sum(dim=1, keepdim=True).shape,
    x.unsqueeze(0).shape)
```

Exercise 3. Predict the output (level 2)

What does this print? NumPy broadcasts like PyTorch.

```
import numpy as np

preds = np.array([[1.0], [2.0], [3.0]])
y = np.array([1.0, 2.0, 3.0])
diff = preds - y
print(diff.shape, round(float((diff ** 2).mean()), 4))
```

Exercise 4. Fill the gap (level 2)

Build float32 features, and a 1-D target from a column that arrives with shape (n, 1).

```
X_t = torch.as_tensor(X, dtype=torch._____(1))
y_t = torch.as_tensor(y, dtype=torch.float32)._____(2)(-1)
```

Exercise 5. Write the function (level 3)

Write `mean_squared_error(preds, y) -> float` with NumPy. `preds` arrives as shape `(n,)` or `(n, 1)`, and `y` as `(n,)`.

- Flatten `preds` with `reshape(-1)` before comparing.
- Raise `ValueError` when the flattened sizes differ, or when there are no rows.
- Return the mean of the squared differences as a Python `float`.

Your code must pass these checks:

- Flatten (n, 1) predictions first, so perfect predictions give 0.0.
- Return the mean of squared differences as a float.
- Raise ValueError when sizes differ or there are no rows.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Mean squared error with predictions flattened to the target's shape.
def mean_squared_error(
    preds: np.ndarray,
    y: np.ndarray,
) -> float: ...
```

Starter code:

```
import numpy as np


def mean_squared_error(preds, y):
```

```
    return float(((preds - y) ** 2).mean())
```

Exercise 6. Spot the bug (level 4)

After one pass through `centre_batches`, the training data itself has changed. Tap the line responsible, then fix it. NumPy slices share memory, as PyTorch slices do.

```
import numpy as np


def centre_batches(X, batch_size):
    batches = []
    for start in range(0, len(X), batch_size):
        batch = X[start:start + batch_size]
        batch -= batch.mean(axis=0)
        batches.append(batch)
    return batches
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: torch.tensor on a float64 array keeps float64; nn.Linear on it raises 'mat1 and mat2 must have the same dtype, but got Double and Float'; from_numpy, view and slices share memory, torch.tensor copies; (3, 1) - (3,) broadcasts to (3, 3); MSELoss on (3, 1) against (3,) gives 1.3333 with a broadcasting warning. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), Python 3.12, November 2026.)
- The E2 and E3 outputs; NumPy slices are views, so -= on a slice changes the source. (verified: Run on PyTorch 2.15.0 and NumPy 2.4.6 and 2.5.4.)

B23-L2 Autograd, gradients from the chain rule

Concept

Training needs the gradient of the loss with respect to every weight. **Autograd** computes it. Create a tensor with `requires_grad=True` and PyTorch records every operation that uses it. `loss.backward()` walks that record backwards with the chain rule and stores each gradient in the tensor's `.grad`. Examples use PyTorch 2.15.

```
w = torch.tensor(3.0, requires_grad=True)
y = w ** 2 + 2 * w
y.backward()
w.grad.item()          # 8.0, since dy/dw = 2w + 2
```

Rules that matter in a training step:

- `backward()` needs a scalar, such as a mean loss. On a vector it raises "grad can be implicitly created only for scalar outputs".
- Tensors you create with `requires_grad=True` are **leaves** (`w.is_leaf` is `True`). Results of operations are not, and only leaves get a `.grad`.
- `.grad` adds up across `backward()` calls. Reset it with `w.grad.zero_()`, or `optimizer.zero_grad()` (lesson B23-L3).
- Update weights in place inside `with torch.no_grad():`, as `w -= lr * w.grad`. Outside that block, the in-place update raises "a leaf Variable that requires grad is being used in an in-place operation". Writing `w = w - lr * w.grad` instead builds a new, non-leaf tensor whose `.grad` stays `None`, and the next step fails with a `TypeError`.
- `preds.detach()` returns a tensor without history, for logging or `.numpy()`.

Check a gradient you derive. For a linear model with mean squared error, the gradient is `2 / n * X.T @ (X @ w - y)`. On a three-row example, autograd and this formula both gave [-17.6667, -22.6667]. To test any gradient formula, use **central differences**: move one weight by a small `h` each way, and `(f(w + h) - f(w - h)) / (2 * h)` approximates its gradient. If the two disagree beyond about 1e-6, the formula is wrong.

Why it matters

A custom loss for a survey model trained, but slowly, and to worse results than plain mean squared error. A central-difference check on five weights showed every gradient was half its true size: a factor of 2 was missing from the hand-written derivative. The check took ten lines and found in minutes what a week of tuning learning rates had not.

Worked example: weak code

```
1 for step in range(100):
2     loss = ((X @ w - y) ** 2).mean()
3     loss.backward()
1     w = w - lr * w.grad
```

1. **Line 4.** Builds a new non-leaf tensor. Its .grad is None, so the second step raises TypeError. The gradient is also never reset.

Worked example: strong code

```
1 import numpy as np
2 import torch
3
4
```

```python
def train_linear(X: torch.Tensor, y: torch.Tensor, lr: float, steps: int) -> torch.Tensor:
    """Fit y = X @ w by gradient descent with autograd.

    Args:
        X: Features, shape (n, k), float32.
        y: Targets, shape (n,).
        lr: Learning rate.
        steps: Number of updates.

    Returns:
        The fitted weights, shape (k,).
    """
    w = torch.zeros(X.shape[1], requires_grad=True)
    for _ in range(steps):
        loss = ((X @ w - y) ** 2).mean()
        loss.backward()
        with torch.no_grad():
            w -= lr * w.grad
        w.grad.zero_()
    return w.detach()


def mse_gradient(X: np.ndarray, y: np.ndarray, w: np.ndarray) -> np.ndarray:
    """The same gradient by hand: 2 / n * X.T @ (X @ w - y)."""
    return 2 / len(y) * X.T @ (X @ w - y)
```

1. **Line 17.** A leaf tensor: backward() fills its .grad.
2. **Line 19.** A scalar, so backward() needs no extra argument.
3. **Line 22.** In place, inside no_grad: w stays the same leaf tensor.
4. **Line 23.** Gradients add up, so reset them after each update.
5. **Line 29.** The formula autograd computes, written out. Use it to check your understanding.

backward() fills .grad on leaf tensors. Update in place under no_grad, reset .grad, and check any hand-written gradient with central differences.

Recap card

- **What backward() does** Applies the chain rule from a scalar loss and stores gradients in .grad of leaf tensors (created with requires_grad=True).
- **Safe manual update** with torch.no_grad(): w -= lr * w.grad, then w.grad.zero_().
- **Gradient of mean squared error for a linear model** 2 / n * X.T @ (X @ w - y).

- **Central-difference check** (f(w + h) - f(w - h)) / (2 * h) per weight. Disagreement beyond about 1e-6 means the formula is wrong.

Practice

Exercise 1. Multiple choice (level 1)

`w -= lr * w.grad` raises "a leaf Variable that requires grad is being used in an in-place operation". What is the fix?

- a) Put the update inside `with torch.no_grad():`
- b) Write `w = w - lr * w.grad` instead
- c) Set `w.requires_grad = False` for the rest of training
- d) Call `loss.backward()` twice

Why?

- r1) The update is not part of the model, so autograd must not record it. no_grad stops the recording and keeps w the same leaf tensor.
- r2) no_grad makes the update run faster on the GPU.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
import torch

w = torch.tensor(3.0, requires_grad=True)
y = w ** 2 + 2 * w
y.backward()
print(w.grad.item(), w.is_leaf, y.is_leaf)
```

Exercise 3. Fill the gap (level 2)

Complete the central-difference estimate for weight `i`, and the safe in-place update.

```
estimate = (f(w_plus) - f(w_minus)) / (_____(1))

with torch._____(2)():
    w -= lr * w.grad
```

Exercise 4. Write the function (level 3)

Write `mse_gradient(X, y, w) -> np.ndarray`, the gradient of `((X @ w - y) ** 2).mean()` with respect to `w`, with NumPy.

- Return `2 / n * X.T @ (X @ w - y)`, where `n` is the number of rows.
- Raise `ValueError` when there are no rows, when `len(y)` differs from the rows of `X`, or when `len(w)` differs from the columns of `X`.

Your code must pass these checks:

- Return 2 / n * X.T @ (X @ w - y).
- At a perfect fit the gradient is zero.
- Raise ValueError for no rows or mismatched shapes.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Gradient of the mean squared error of a linear model.
def mse_gradient(
    X: np.ndarray,
    y: np.ndarray,
    w: np.ndarray,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def mse_gradient(X, y, w):
    return X.T @ (X @ w - y)
```

Exercise 5. Write the function (level 3)

Write `numerical_gradient(f, w, h=1e-5) -> np.ndarray`, a central-difference estimate of the gradient of `f` at `w`.

- For each index `i`, copy `w` twice, add `h` to element `i` of one copy and subtract `h` from the other, and use `(f(plus) - f(minus)) / (2 * h)`.
- Leave `w` unchanged, and return float values even when `w` holds integers.

What you need to know.

`arr.astype(float)` returns a float copy of an array, and `np.zeros_like(arr)` returns zeros with its shape and dtype:

```
counts = np.array([3, 5])
counts.astype(float)          # array([3., 5.])
np.zeros_like(np.ones(2))     # array([0., 0.])
```

To type a parameter that is a function, use `Callable` from `collections.abc`: `Callable[[str], int]` means "takes a str, returns an int", as `len` does.

Your code must pass these checks:

- Use (f(plus) - f(minus)) / (2 * h) for each element.
- Work on float copies: leave w unchanged, and handle integer input.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Central-difference estimate of the gradient of f at w.
```

```
def numerical_gradient(
    f: Callable[[np.ndarray], float],
    w: np.ndarray,
    h: float = 1e-05,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def numerical_gradient(f, w, h=1e-5):
    return np.zeros(len(w))
```

Exercise 6. Spot the bug (level 4)

This manual training loop raises `TypeError: unsupported operand type(s) for *: 'float' and 'NoneType'` on its second step. Tap the line responsible, then fix it.

```
import torch

w = torch.zeros(X.shape[1], requires_grad=True)
for step in range(100):
    loss = ((X @ w - y) ** 2).mean()
    loss.backward()
    w = w - lr * w.grad
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: d(w^2 + 2w)/dw at 3 is 8.0; backward on a vector raises 'grad can be implicitly created only for scalar outputs'; an in-place update of a leaf outside no_grad raises 'a leaf Variable that requires grad is being used in an in-place operation'; w = w - lr * w.grad leaves a non-leaf whose .grad is None and the next step raises TypeError; autograd and 2/n X^T(Xw - y) both give [-17.6667, -22.6667]. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), Python 3.12, and NumPy, November 2026.)

B23-L3 Gradient descent, learning rates and optimisers

Concept

Gradient descent moves every weight a small step against its gradient: `w = w - lr * grad`. The **learning rate** `lr` sets the step size, and it decides whether training works at all.

On `f(p) = p ** 2` the gradient is `2 * p`, so one step multiplies `p` by `1 - 2 * lr`. Starting at 4, after five steps:

- `lr = 0.1` gives 1.3107, closing in on the minimum at 0.
- `lr = 0.9` gives -1.3107: it jumps across the minimum on every step.
- `lr = 1.1` gives -9.9533: each jump overshoots further, and training diverges.

In a real model, a loss that grows, or turns into `nan`, within a few steps usually means the rate is too high. A loss that barely falls suggests it is too low.

Scale the features (lesson B23-L6). One rate serves every weight. A feature in thousands gets large gradients and needs small steps, and a 0 to 1 feature needs large ones. Standardised features let one rate suit both.

Mini-batches. Stochastic gradient descent computes each step's gradient on a batch of rows, not the whole dataset: noisier, far cheaper, and the standard choice.

Optimisers hold the update rule (PyTorch 2.15):

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

optimizer.zero_grad()    # reset .grad on every parameter
loss.backward()          # fill .grad (lesson B23-L2)
optimizer.step()         # apply the update
```

`torch.optim.SGD(params, lr=0.1, momentum=0.9)` adds **momentum**, a running direction that smooths noisy batch gradients. **AdamW** also scales each weight's step by its recent gradient size, so it is less sensitive to the rate. Its `weight_decay`, which pulls weights towards zero as regularisation, defaults to 0.01. SGD's defaults to 0. Start with AdamW at 1e-3, then tune the rate on validation loss.

Why it matters

A first neural model of survey spend produced `nan` loss after three batches. The team suspected the data and spent a day hunting for missing values. The features held income in pounds next to 0 to 1 flags, and the learning rate was 0.1. Standardising the features and starting AdamW at 1e-3 gave a falling loss on the first run.

Worked example: weak code

```
import numpy as np

w = np.zeros(X.shape[1])
```

```
for _ in range(1000):
    w = w - 0.5 * 2 / len(y) * X.T @ (X @ w - y)
```

1. **Line 4.** No record of the loss, so divergence goes unseen until the weights are huge.
2. **Line 5.** A hardcoded rate with unscaled features. Whether it converges depends on the data.

Worked example: strong code

```
import numpy as np


def gradient_descent(
    X: np.ndarray, y: np.ndarray, lr: float, steps: int
) -> tuple[np.ndarray, list[float]]:
    """Fit a linear model by full-batch gradient descent on mean squared error.

    Args:
        X: Standardised features, shape (n, k).
        y: Targets, shape (n,).
        lr: Learning rate, from config.
        steps: Number of updates.

    Returns:
        The weights and the loss before each update.

    Raises:
        ValueError: If lr is not positive or steps is below 1.
    """
    if lr <= 0 or steps < 1:
        raise ValueError(f"need lr > 0 and steps >= 1, got {lr} and {steps}")
    w = np.zeros(X.shape[1])
    losses: list[float] = []
    for _ in range(steps):
        losses.append(float(((X @ w - y) ** 2).mean()))
        w = w - lr * 2 / len(y) * X.T @ (X @ w - y)
    return w, losses
```

1. **Line 10.** Scaling comes first, so one rate suits every weight.
2. **Line 22.** A zero or negative rate never trains. Fail fast.
3. **Line 26.** Record the loss every step. A rising curve is the first sign of a bad rate.
4. **Line 27.** Step against the gradient: minus, not plus.

Step against the gradient, scale features so one rate fits all weights, watch the loss curve, and let an optimiser such as AdamW hold the update rule.

Recap card

- **The gradient descent update** w = w - lr * grad. Against the gradient, scaled by the learning rate.
- **Signs of a bad learning rate** Too high: loss grows or turns into nan. Too low: loss barely falls.
- **One optimiser step in PyTorch** optimizer.zero_grad(), loss.backward(), optimizer.step().
- **AdamW and SGD defaults in PyTorch 2.15** AdamW weight_decay 0.01, SGD weight_decay 0. A good start: AdamW with lr 1e-3.

Practice

Exercise 1. Multiple choice (level 1)

Training loss rises for three batches, then prints `nan`. The features are unscaled and the learning rate is 0.1. What do you try first?

- a) Standardise the features and lower the learning rate
- b) Train for more epochs
- c) Add more layers
- d) Increase the learning rate to escape the bad region

Why?

- r1) A loss that grows each step is overshooting: steps too large for the gradients that unscaled features produce.
- r2) Smaller numbers take less memory.

Exercise 2. Predict the output (level 2)

What does this print? Gradient descent on `p ** 2`, whose gradient is `2 * p`.

```
for lr in [0.1, 0.9, 1.1]:
    p = 4.0
    for _ in range(5):
        p = p - lr * 2 * p
    print(lr, round(p, 4))
```

Exercise 3. Fill the gap (level 2)

Build an AdamW optimiser over the model's weights, and complete one update.

```
optimizer = torch.optim._____(1)(model.parameters(), lr=1e-3)

optimizer.zero_grad()
loss = loss_fn(model(x), y)
```

```
loss.backward()
optimizer._____(2)()
```

Exercise 4. Predict the output (level 3)

What does this print? (PyTorch 2.15)

```
import torch

p = torch.tensor([4.0], requires_grad=True)
optimizer = torch.optim.SGD([p], lr=0.1)
for _ in range(3):
    optimizer.zero_grad()
    (p ** 2).sum().backward()
    optimizer.step()
print(round(p.item(), 3))
```

Exercise 5. Write the function (level 3)

Write `gradient_descent(X, y, lr, steps) -> tuple[np.ndarray, list[float]]` for a linear model with mean squared error.

- Raise `ValueError` unless `lr > 0` and `steps >= 1`.
- Start from zero weights. On each step, append the current loss `((X @ w - y) ** 2).mean()` as a float, then update `w = w - lr * 2 / n * X.T @ (X @ w - y)`.
- Return the final weights and the list of losses.

Your code must pass these checks:

- Step against the gradient, w - lr * grad, from zero weights.
- Record the loss before each update, as a float: the first is the loss at zero weights.
- Raise ValueError unless lr > 0 and steps >= 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Fit a linear model by full-batch gradient descent on mean squared error.
def gradient_descent(
    X: np.ndarray,
    y: np.ndarray,
    lr: float,
    steps: int,
) -> tuple[np.ndarray, list[float]]: ...
```

Starter code:

```
import numpy as np


def gradient_descent(X, y, lr, steps):
```

```
    return np.zeros(X.shape[1]), []
```

Exercise 6. Spot the bug (level 4)

The loss rises on every step, even with a learning rate of 0.001. Tap the line responsible, then fix it.

```
import numpy as np


def gradient_descent(X, y, lr, steps):
    w = np.zeros(X.shape[1])
    losses = []
    for _ in range(steps):
        losses.append(float(((X @ w - y) ** 2).mean()))
        grad = 2 / len(y) * X.T @ (X @ w - y)
        w = w + lr * grad
    return w, losses
```

Write down the line number of the bug. Then write the fix.

Sources

- Gradient descent on p^2 from 4 for five steps: 1.3107 at lr 0.1, -1.3107 at 0.9, -9.9533 at 1.1; torch.optim.SGD with lr 0.1 gives 2.048 after three steps; AdamW default weight_decay is 0.01 and SGD's is 0. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU) and Python 3.11 and 3.12, November 2026.)

B23-L4 Models as nn.Module, layers, activations and parameters

Concept

A model is a class that subclasses `nn.Module` (PyTorch 2.15):

```
class Recommender(nn.Module):
    def __init__(self, n_features: int, hidden: int) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

- Call `super().__init__()` first. Without it, assigning a layer raises "cannot assign module before Module.**init**() call".

- Build layers in `__init__`, use them in `forward`, and call the model as `model(x)`, not `model.forward(x)`, so PyTorch's hooks run.
- `nn.Linear(in_features, out_features)` computes `x @ W.T + b`. Its weight has shape `(out, in)`: `nn.Linear(3, 2)` holds a (2, 3) weight and 2 biases, 8 parameters in all. Count a model's parameters with `sum(p.numel() for p in model.parameters())`. The model above, with 10 features and 16 hidden units, has 193.
- **Activations make depth useful.** Two `nn.Linear` layers in a row equal one linear layer, because a product of matrices is a matrix. `nn.ReLU()`, which replaces negatives with 0, between them lets the model fit curves and interactions.
- **Register every layer.** `parameters()` finds layers stored as attributes, or inside `nn.Sequential` or `nn.ModuleList`. Layers kept in a plain Python list are invisible to it. The optimiser never updates them, and `model.to(device)` leaves them behind.

`model.train()` and `model.eval()` set the `training` flag that dropout reads (lesson B16-L2).

In NumPy, the same forward pass is `np.maximum(X @ W1.T + b1, 0) @ W2.T + b2`. On random weights it matched `nn.Sequential` to within 1e-6. Writing it once by hand shows there is nothing hidden in a layer.

Why it matters

A model with layer sizes [20, 64, 32, 1] built its hidden layers from a config list, `self.hidden = [nn.Linear(a, b) for a, b in pairs]`, and kept the output layer as its own attribute. Training ran for an hour and the loss barely moved. `sum(p.numel() for p in model.parameters())` printed 33: only the output layer was registered, and the optimiser had never seen the hidden layers. `nn.ModuleList` registered all 3,457 parameters, and the loss fell from the first epoch.

Worked example: weak code

```
class Recommender(nn.Module):
    def __init__(self, sizes):
        super().__init__()
        self.layers = [nn.Linear(a, b) for a, b in zip(sizes, sizes[1:])]

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
```

```
9         return x
```

1. **Line 4.** A plain list: parameters() never sees these layers, so they never train.
2. **Line 8.** No activation between layers, so the stack is one linear map.

Worked example: strong code

```
1  import torch
2  from torch import nn
3
4
5  class Recommender(nn.Module):
6      """Feed-forward network for a binary survey outcome. Returns
       logits."""
7
8      def __init__(self, sizes: list[int]) -> None:
9          """Build Linear layers between consecutive sizes.
10
11         Args:
12             sizes: Layer widths from input to output, such as [10, 16, 1].
13
14         Raises:
15             ValueError: If fewer than two sizes are given.
16         """
(1)        super().__init__()
18         if len(sizes) < 2:
19             raise ValueError(f"need at least input and output sizes, got
       {sizes}")
(2)        self.layers = nn.ModuleList(nn.Linear(a, b) for a, b in
       zip(sizes, sizes[1:]))
21
22     def forward(self, x: torch.Tensor) -> torch.Tensor:
23         """Apply each layer, with ReLU between them."""
24         for layer in self.layers[:-1]:
(3)            x = torch.relu(layer(x))
(4)        return self.layers[-1](x)
27
28
29 def n_parameters(model: nn.Module) -> int:
30     """Number of trainable values in a model."""
(5)    return sum(p.numel() for p in model.parameters())
```

1. **Line 17.** First line of every module's **init**.
2. **Line 20.** ModuleList registers every layer, so the optimiser and .to() see them.
3. **Line 25.** ReLU between layers makes the stack non-linear.
4. **Line 26.** No activation on the output: the loss takes raw scores (lesson B23-L5).
5. **Line 31.** Log this with every run. A surprising count is the fastest check that layers are registered.

Subclass nn.Module, call `super().__init__()` first, register layers in attributes, Sequential or ModuleList, and put activations between Linear layers.

Recap card

- **Skeleton of a PyTorch model** Subclass nn.Module, call `super().__init__()` first, build layers in `__init__`, use them in forward, call model(x).
- **nn.Linear(in, out) weight shape and parameter count** Weight (out, in), bias (out,). Parameters: in * out + out.
- **Why put ReLU between Linear layers** Two linear layers equal one. A non-linear activation lets depth fit curves and interactions.
- **Layers in a plain Python list** Not registered: parameters() misses them and they never train. Use nn.ModuleList or nn.Sequential.

Practice

Exercise 1. Multiple choice (level 1)

A model is `nn.Sequential(nn.Linear(10, 64), nn.Linear(64, 1))`. Why does it fit no better than `nn.Linear(10, 1)`?

- a) Two linear layers in a row equal one linear layer. It needs an activation such as nn.ReLU() between them
- b) 64 hidden units is too few
- c) Sequential does not train its second layer
- d) The output needs 10 units, one per feature

Why?

- r1) x @ W1.T @ W2.T is x times one combined matrix, so depth adds nothing without a non-linear step.
- r2) ReLU makes training faster on the GPU.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
1 from torch import nn
2
3 model = nn.Sequential(nn.Linear(10, 16), nn.ReLU(), nn.Linear(16, 1))
4 print(sum(p.numel() for p in model.parameters()), nn.Linear(3,
    2).weight.shape)
```

Exercise 3. Fill the gap (level 2)

Complete a module whose layers come from a list, so every layer is registered.

```
class Stack(nn.Module):
    def __init__(self, sizes: list[int]) -> None:
        super().____(1)()
        self.layers = nn.____(2)(nn.Linear(a, b) for a, b in zip(sizes,
    sizes[1:]))
```

Exercise 4. Write the function (level 3)

Write `mlp_forward(X, W1, b1, W2, b2) -> np.ndarray`, the forward pass of `nn.Sequential(nn.Linear, nn.ReLU, nn.Linear)` in NumPy, with weights in PyTorch's `(out, in)` layout.

- Return `np.maximum(X @ W1.T + b1, 0) @ W2.T + b2`.
- Raise `ValueError` when the columns of `X` differ from the columns of `W1`, when the columns of `W2` differ from the rows of `W1`, or when a bias length differs from its weight's rows.

What you need to know.

`np.maximum(a, 0)` compares element by element and keeps the larger value, so it replaces negatives with 0:

```
np.maximum(np.array([-2.0, 0.5, 3.0]), 0)    # array([0. , 0.5, 3. ])
```

Your code must pass these checks:

- Compute np.maximum(X @ W1.T + b1, 0) @ W2.T + b2, with weights in (out, in) layout.
- Apply ReLU to the hidden layer only, before the second Linear.
- Raise ValueError when any weight or bias shape does not fit.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Forward pass of Linear, ReLU, Linear with PyTorch's (out, in) weights.
def mlp_forward(
    X: np.ndarray,
    W1: np.ndarray,
    b1: np.ndarray,
    W2: np.ndarray,
    b2: np.ndarray,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


```

```
def mlp_forward(X, W1, b1, W2, b2):
    return X @ W1 @ W2
```

Exercise 5. Write the function (level 3)

Write `count_linear_params(sizes: list[int]) -> int`: the number of parameters in a stack of `nn.Linear` layers between consecutive sizes, such as `[10, 16, 1]`.

- Each `nn.Linear(a, b)` holds `a * b` weights and `b` biases.
- Raise `ValueError` when there are fewer than two sizes, or when any size is below 1.

Your code must pass these checks:

- Sum a * b + b over consecutive pairs of sizes.
- Raise ValueError for fewer than two sizes, or any size below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Parameters in Linear layers between consecutive sizes.
def count_linear_params(sizes: list[int]) -> int: ...
```

Starter code:

```
def count_linear_params(sizes):
    return 0
```

Exercise 6. Spot the bug (level 4)

`sum(p.numel() for p in model.parameters())` prints 0, and the optimiser raises "optimizer got an empty parameter list". Tap the line responsible, then fix it.

```
import torch
from torch import nn


class Stack(nn.Module):
    def __init__(self, sizes):
        super().__init__()
        self.layers = [nn.Linear(a, b) for a, b in zip(sizes, sizes[1:])]

    def forward(self, x):
        for layer in self.layers[:-1]:
            x = torch.relu(layer(x))
        return self.layers[-1](x)
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: Linear(3, 2) weight is (2, 3) with 8 parameters; Sequential(Linear(10, 16), ReLU, Linear(16, 1)) has 193; layers in a plain list give 0 parameters and ModuleList gives 4 for two layers; a missing super().**init**() raises 'cannot assign module before Module.**init**() call'; two stacked Linear layers equal one combined linear map; the NumPy forward pass matched nn.Sequential within 1e-6. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), Python 3.12, November 2026.)

B23-L5 Loss functions, logits and target shapes

Concept

The loss turns predictions and targets into one number to minimise. Choose it from the target, and match the shapes and dtypes it expects exactly. Examples use PyTorch 2.15.

Numeric target: `nn.MSELoss()`**.** Predictions and targets need the same shape. A model ending in `nn.Linear(hidden, 1)` returns `(n, 1)`, and against targets of shape `(n,)` the loss broadcasts to `(n, n)` (lesson B23-L1). PyTorch warns, "Using a target size ... that is different to the input size", and returns the wrong value: 1.3333 for three perfect predictions instead of 0.0. Squeeze first: `loss_fn(model(x).squeeze(1), y)`.

Binary target: `nn.BCEWithLogitsLoss()`**.** The model outputs a **logit**, a raw score with no sigmoid applied. The loss applies the sigmoid itself, in a numerically stable form. In float32, `torch.log(torch.sigmoid(torch.tensor([-100.0])))` gives `-inf`, while the loss for that logit with target 1 is the correct 100.0. Two rules:

- Do not end the model with `nn.Sigmoid()`. The loss would apply a second sigmoid. For logits [2, -1, 0] and targets [1, 0, 1] it returned 0.5525 instead of 0.3778, and the gradients shrink.
- Targets are floats with the logits' shape. Integer targets raise "result type Float can't be cast to the desired output type Long".

Apply `torch.sigmoid(logits)` only when you need probabilities, at prediction time.

Imbalanced binary target.

`nn.BCEWithLogitsLoss(pos_weight=torch.tensor([n_neg / n_pos]))` scales the loss on positive rows, so the rare class carries as much total weight as the common one.

Several classes: `nn.CrossEntropyLoss()` takes logits of shape `(n, classes)` and integer class indices of shape `(n,)` (lesson B16-L2).

The stable binary loss, written out for logits `z` and targets `y`, is the mean of `max(z, 0) - z * y + log(1 + exp(-|z|))`. It never takes the logarithm of a number that has rounded to zero.

Why it matters

A churn model's validation loss was stuck near 0.69, the loss of predicting 0.5 for everyone, while the training loss crept down. The model ended in `nn.Sigmoid()` and trained with `BCEWithLogitsLoss`. Outputs were squeezed between 0.5 and 0.73 by the second sigmoid, and gradients were a fraction of their size. Removing the sigmoid layer took validation loss from 0.69 to 0.41 in the same number of epochs.

Worked example: weak code

```
❶ model = nn.Sequential(nn.Linear(k, 16), nn.ReLU(), nn.Linear(16, 1),
    nn.Sigmoid())
  loss_fn = nn.BCEWithLogitsLoss()
❷ loss = loss_fn(model(x), torch.as_tensor(y))
```

1. **Line 1.** Sigmoid in the model and again in the loss: wrong values, weak gradients.
2. **Line 3.** (n, 1) outputs against (n,) targets, and integer y raises a dtype error.

Worked example: strong code

```python
import numpy as np
import torch
from torch import nn


def binary_loss(y_train: np.ndarray) -> nn.BCEWithLogitsLoss:
    """Binary loss on logits, weighted for class imbalance.

    Args:
        y_train: Training targets of 0 and 1.

    Returns:
        BCEWithLogitsLoss with pos_weight = negatives / positives.

    Raises:
        ValueError: If the training data has no positive rows.
    """
    n_pos = int(y_train.sum())
    if n_pos == 0:
        raise ValueError("no positive rows in the training data")
```

```
    return nn.BCEWithLogitsLoss(pos_weight=torch.tensor([(len(y_train) - n_pos) / n_pos]))


def batch_loss(model: nn.Module, loss_fn: nn.Module, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """Loss on one batch: logits squeezed to (n,), targets as float."""
    logits = model(x).squeeze(1)
    return loss_fn(logits, y.float())


def predict_proba(model: nn.Module, x: torch.Tensor) -> torch.Tensor:
    """Probabilities, for reporting. Sigmoid is applied here, not in the model."""
    with torch.no_grad():
        return torch.sigmoid(model(x).squeeze(1))
```

1. **Line 21.** Weight from training rows only, so the rare class counts as much as the common one.
2. **Line 26.** (n, 1) becomes (n,): the target's shape.
3. **Line 27.** Float targets, raw logits: what the loss expects.
4. **Line 33.** Sigmoid only where probabilities are needed.

Regression: MSELoss on matching shapes. Binary: logits into BCEWithLogitsLoss with float targets and no sigmoid in the model. Squeeze (n, 1) outputs first.

Recap card

- **Loss for a numeric target** nn.MSELoss() on predictions and targets of the same shape. Squeeze (n, 1) outputs first.
- **Loss for a 0 or 1 target** nn.BCEWithLogitsLoss() on raw logits and float targets. No nn.Sigmoid() in the model.
- **pos_weight for an imbalanced binary target** torch.tensor([n_neg / n_pos]), computed on training rows.
- **Stable binary loss formula** mean(max(z, 0) - z * y + log1p(exp(-|z|))).

Practice

Exercise 1. Multiple choice (level 1)

You predict whether a respondent would recommend the brand (0 or 1). Which output layer and loss do you use?

- a) `nn.Linear(hidden, 1)` with no activation, and `nn.BCEWithLogitsLoss()` on float targets
- b) `nn.Linear(hidden, 1)` then `nn.Sigmoid()`, and `nn.BCEWithLogitsLoss()`

- c) `nn.Linear(hidden, 1)` and `nn.MSELoss()` on the 0 and 1 targets
- d) `nn.Linear(hidden, 1)` and `nn.CrossEntropyLoss()` on float targets

Why?

- r1) BCEWithLogitsLoss applies the sigmoid itself, stably, so the model outputs raw logits.
- r2) A model without a final activation trains faster on the GPU.

Exercise 2. Predict the output (level 2)

What does this print? Ignore the warning PyTorch writes to stderr. (PyTorch 2.15)

```
import torch
from torch import nn

pred = torch.tensor([[1.0], [2.0], [3.0]])
target = torch.tensor([1.0, 2.0, 3.0])
loss_fn = nn.MSELoss()
print(round(loss_fn(pred, target).item(), 4), loss_fn(pred.squeeze(1),
    target).item())
```

Exercise 3. Fill the gap (level 2)

Weight the rare positive class, and give the loss logits in the target's shape.

```
loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([_____(1)]))
loss = loss_fn(model(x)._____(2)(1), y.float())
```

Exercise 4. Write the function (level 3)

Write `bce_with_logits(logits, targets) -> float` with NumPy, in the stable form.

- Return the mean of `max(z, 0) - z * y + log1p(exp(-abs(z)))` over all rows.
- Raise `ValueError` when the shapes differ or there are no rows.

What you need to know.

`np.abs`, `np.exp` and `np.log1p` work element by element. `np.log1p(x)` computes `log(1 + x)`, and stays accurate when `x` is tiny:

```
np.abs(np.array([-2.0, 3.0]))        # array([2., 3.])
np.log1p(np.array([0.0, 1e-20]))     # array([0.e+00, 1.e-20])
```

`np.maximum(a, 0)` keeps the larger of each value and 0.

Your code must pass these checks:

- Average max(z, 0) - z * y + log1p(exp(-abs(z))).
- Stay finite for large logits: no log of 0, no overflow in exp.

- Raise ValueError when shapes differ or there are no rows.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Binary cross-entropy on logits, in the numerically stable form.
def bce_with_logits(
    logits: np.ndarray,
    targets: np.ndarray,
) -> float: ...
```

Starter code:

```python
import numpy as np


def bce_with_logits(logits, targets):
    p = 1 / (1 + np.exp(-logits))
    return float(-np.mean(targets * np.log(p) + (1 - targets) * np.log(1 - p)))
```

Exercise 5. Write the function (level 3)

Write `positive_weight(y_train) -> float`: the `pos_weight` for `BCEWithLogitsLoss`, negatives divided by positives.

- Raise `ValueError` when there are no rows, no positives, or any value other than 0 and 1.

What you need to know.

`np.isin(a, values)` tests each element for membership, and `.all()` is `True` only when every test passes:

```python
np.isin(np.array([1, 5]), [1, 2]).all()    # False
```

Your code must pass these checks:

- Return negatives / positives as a float.
- Raise ValueError for no rows, no positives, or values other than 0 and 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# pos_weight for BCEWithLogitsLoss: negatives / positives.
def positive_weight(y_train: np.ndarray) -> float: ...
```

Starter code:

```python
import numpy as np


def positive_weight(y_train):
    return 1.0
```

Exercise 6. Spot the bug (level 4)

Validation loss sits near 0.69 for every epoch, and predicted probabilities never fall below 0.5. Tap the line responsible, then fix it.

```
from torch import nn


class Recommender(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_features, 16), nn.ReLU(),
    nn.Linear(16, 1), nn.Sigmoid())

    def forward(self, x):
        return self.net(x).squeeze(1)


loss_fn = nn.BCEWithLogitsLoss()
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: MSELoss on (3, 1) against (3,) returns 1.3333 with a size warning, 0.0 after squeeze(1); BCEWithLogitsLoss on logits [2, -1, 0] and targets [1, 0, 1] is 0.3778 and 0.5525 on their sigmoids; long targets raise 'result type Float can't be cast to the desired output type Long'; in float32 log(sigmoid(-100)) is -inf and the loss for that logit with target 1 is 100.0; Sequential with ReLU outputs and a final sigmoid feeding BCEWithLogitsLoss stays in (0.5, 0.73). (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), Python 3.12, and NumPy, November 2026.)

B23-L6 Dataset, DataLoader and preprocessing fitted on training rows

Concept

Data reaches a model in batches. PyTorch 2.15 splits that job in two:

```
class SurveyRows(Dataset):
    def __init__(self, X: np.ndarray, y: np.ndarray) -> None:
        self.X = torch.as_tensor(X, dtype=torch.float32)
        self.y = torch.as_tensor(y, dtype=torch.float32)

    def __len__(self) -> int:
        return len(self.y)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
```

```
        return self.X[i], self.y[i]


train_loader = DataLoader(train_rows, batch_size=64, shuffle=True)
val_loader = DataLoader(val_rows, batch_size=256)
```

- A `Dataset` reports how many rows it holds (`__len__`) and returns one row (`__getitem__`). Convert to tensors once, in `__init__`, not on every row.
- A `DataLoader` stacks rows into batches. With 10 rows and `batch_size=4` the batches hold 4, 4 and 2 rows. `drop_last=True` drops the short one.
- Shuffle training batches, so each epoch meets the rows in a new order. Leave validation unshuffled: order cannot change its metric, and a fixed order makes runs easier to compare. Seed the shuffle (lesson B16-L3).

Preprocess before the tensors, fitted on training rows only (lesson B10-L2):

- Standardise each feature with the training mean and standard deviation. Replace a standard deviation of 0 with 1, or a constant column divides by zero.
- Apply those training statistics, unchanged, to validation and test rows. Statistics computed on every row leak held-out data into training.
- Impute missing values first. One `NaN` in a batch makes the loss `nan`, and then every weight.
- One-hot encode categories with the categories seen in training (lesson B11-L2).

In NumPy, a shuffled split into batches is `order = rng.permutation(n)`, then `order[i:i + batch_size]` for `i` in `range(0, n, batch_size)`. That is what a shuffling `DataLoader` does with indices.

Why it matters

A model's validation accuracy was 0.86 in the notebook and 0.79 on the next wave's data. The features had been standardised on all 6,000 rows before the train and validation split, and income in the validation rows was higher than in training. Refitting the scaler on the 4,800 training rows only gave a validation accuracy of 0.80, an honest preview of the next wave.

Worked example: weak code

```
X = (X - X.mean(axis=0)) / X.std(axis=0)
X_train, X_val = X[:4800], X[4800:]
```

```
❸ val_loader = DataLoader(SurveyRows(X_val, y_val), batch_size=256,
    shuffle=True)
```

1. **Line 1.** Statistics from every row, including validation: leakage. A constant column divides by zero.
2. **Line 2.** An unshuffled split takes the last rows, which may be the latest wave.
3. **Line 3.** Shuffling validation buys nothing.

Worked example: strong code

```
 1 import numpy as np
 2
 3
 4 def standardise(X_train: np.ndarray, X_other: np.ndarray) ->
     tuple[np.ndarray, np.ndarray]:
 5     """Scale features with statistics from the training rows only.
 6
 7     Args:
 8         X_train: Training features, shape (n, k), no missing values.
 9         X_other: Validation or test features, shape (m, k).
10
11     Returns:
12         Both arrays, scaled with the training mean and standard deviation.
13
14     Raises:
15         ValueError: If either array holds NaN or has no rows.
16     """
17     if len(X_train) == 0 or np.isnan(X_train).any() or
     np.isnan(X_other).any():
❶        raise ValueError("impute missing values first, and pass at least
     one training row")
❷     mean = X_train.mean(axis=0)
20     std = X_train.std(axis=0)
❸     std = np.where(std == 0, 1.0, std)
❹     return (X_train - mean) / std, (X_other - mean) / std
23
24
25 def batch_indices(n: int, batch_size: int, rng: np.random.Generator) ->
     list[np.ndarray]:
26     """Shuffled row indices split into batches, as a shuffling DataLoader
     does."""
❺     order = rng.permutation(n)
28     return [order[i : i + batch_size] for i in range(0, n, batch_size)]
```

1. **Line 18.** A NaN would turn the loss into nan. Stop it here.
2. **Line 19.** Training rows only.
3. **Line 21.** A constant column stays constant instead of dividing by zero.
4. **Line 22.** The same training statistics for both arrays.
5. **Line 27.** A seeded generator makes the batch order repeatable.

A Dataset returns rows, a DataLoader batches them, training shuffles and validation does not, and every preprocessing statistic comes from training rows only.

Recap card

- **The two methods of a Dataset** `__len__` returns the row count. `__getitem__(i)` returns row i as tensors, converted once in `__init__`.
- **DataLoader batches for 10 rows, batch_size=4** 4, 4 and 2. drop_last=True gives 4 and 4.
- **Which loaders shuffle** Training: shuffle=True with a seeded generator. Validation and test: no shuffle.
- **Preprocessing rule** Fit means, standard deviations and categories on training rows only. Guard std 0, impute NaN first.

Practice

Exercise 1. Multiple choice (level 1)

Why does the training `DataLoader` use `shuffle=True` while the validation `DataLoader` does not?

- a) Training benefits from a new row order each epoch. Validation loss does not depend on order, so shuffling it adds nothing
- b) Shuffling validation would leak the targets
- c) Validation must be shuffled too, or its loss is biased
- d) DataLoader cannot shuffle more than one loader

Why?

- r1) Varied batches give each epoch different gradient steps. A validation metric averages over all rows, whatever their order.
- r2) Shuffled loaders run faster.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

What you need to know.

`TensorDataset(X, y)` is a ready-made `Dataset` over tensors with the same number of rows. Row `i` is `(X[i], y[i])`, and its length is the row count.

```
1 import torch
2 from torch.utils.data import DataLoader, TensorDataset
3
```

```
rows = TensorDataset(torch.arange(10.0).reshape(10, 1),
    torch.arange(10.0))
print([len(xb) for xb, _ in DataLoader(rows, batch_size=4)])
print([len(xb) for xb, _ in DataLoader(rows, batch_size=4,
    drop_last=True)])
```

Exercise 3. Fill the gap (level 2)

Complete the two methods every `Dataset` needs.

```
class SurveyRows(Dataset):
    def __init__(self, X, y):
        self.X = torch.as_tensor(X, dtype=torch.float32)
        self.y = torch.as_tensor(y, dtype=torch.float32)

    def _____(1)(self):
        return len(self.y)

    def _____(2)(self, i):
        return self.X[i], self.y[i]
```

Exercise 4. Write the function (level 3)

Write `standardise(X_train, X_other) -> tuple[np.ndarray, np.ndarray]`.

- Compute the mean and standard deviation of each column of `X_train` only (`axis=0`). Replace a standard deviation of 0 with 1.
- Return both arrays scaled as `(X - mean) / std` with those training statistics.
- Raise `ValueError` when `X_train` has no rows, or when either array holds `NaN`.

What you need to know.

`np.isnan(a)` marks each missing value, and `.any()` is `True` when at least one is marked:

```
np.isnan(np.array([1.0, np.nan])).any()    # True
```

`np.where(condition, a, b)` takes `a` where the condition holds and `b` elsewhere.

Your code must pass these checks:

- Use the training mean and standard deviation for both arrays.
- Replace a standard deviation of 0 with 1.
- Raise ValueError for no training rows or any NaN.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Scale features with statistics from the training rows only.
```

```python
def standardise(
    X_train: np.ndarray,
    X_other: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]: ...
```

Starter code:

```python
import numpy as np


def standardise(X_train, X_other):
    both = np.vstack([X_train, X_other])
    mean, std = both.mean(axis=0), both.std(axis=0)
    return (X_train - mean) / std, (X_other - mean) / std
```

Exercise 5. Write the function (level 3)

Write `batch_indices(n, batch_size, rng) -> list[np.ndarray]`, the row order a shuffling `DataLoader` uses.

- Take one `rng.permutation(n)` and split it in order into batches of `batch_size`. The last batch may be shorter.
- Raise `ValueError` when `n` or `batch_size` is below 1.

What you need to know.

`rng.permutation(n)` returns the numbers 0 to n - 1 in a random order, drawn from the generator `rng` (lesson B6-L1):

```python
rng = np.random.default_rng(0)
rng.permutation(5)    # the numbers 0 to 4, shuffled
```

Your code must pass these checks:

- Split one permutation into batches of batch_size, with a shorter last batch.
- Use one rng.permutation(n), split in order.
- Raise ValueError when n or batch_size is below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Shuffled row indices split into batches, as a shuffling DataLoader does.
def batch_indices(
    n: int,
    batch_size: int,
    rng: np.random.Generator,
) -> list[np.ndarray]: ...
```

Starter code:

```python
import numpy as np


def batch_indices(n, batch_size, rng):
```

```
    return [np.arange(n)]
```

Exercise 6. Spot the bug (level 4)

Validation accuracy is higher in development than on the next wave's data. Tap the line that lets validation rows shape the training data, then fix it.

```
import numpy as np


def prepare(X, y, rng, n_train):
    X = (X - X.mean(axis=0)) / X.std(axis=0)
    order = rng.permutation(len(X))
    train, val = order[:n_train], order[n_train:]
    return X[train], y[train], X[val], y[val]
```

Write down the line number of the bug. Then write the fix.

Practical: Train a PyTorch model on survey data

Platform: Laptop. Work in `labs/torch/` in this repository. It holds a seeded generator of survey data, a training script and a logistic regression baseline. You write the steps from module B23 and from lessons B16-L2 and B16-L3: preprocessing, the dataset, the model, one training epoch, evaluation and early stopping. You need PyTorch 2.15 for the laptop: it is not in the app or the `dev` group. When the model beats the baseline, push the lab to your own GitHub repository with a README: it is the roadmap's PyTorch evidence for month 3.

1. Run `cd labs/torch`, create a virtual environment, then `pip install -e .` and `pip install pytest`. If the PyTorch wheel is large for your machine, install the CPU build first: `pip install torch --index-url https://download.pytorch.org/whl/cpu`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/survey_torch/`.
2. In `src/survey_torch/prep.py`, write `split_indices` and `Standardiser` (lessons B23-L6 and B10-L2). Run `pytest tests/test_prep.py -k "split or standard"`.
3. Write `EarlyStopping` in the same file (lesson B16-L2). It must keep a deep copy of the best state.
4. In `src/survey_torch/nn_steps.py`, write `SurveyRows` and `make_model` (lessons B23-L1, B23-L4 and B23-L6), then `train_one_epoch` and `evaluate` (lessons B23-L3, B23-L5 and B16-L2). Run `pytest`: every test passes.
5. Run `python -m survey_torch train`. It prints the baseline's and your model's validation loss and accuracy. Change `--hidden`, `--lr` and `--patience`, and keep notes on what each change did.

6. Run the same command twice with the same `--seed` and check the numbers match (lesson B16-L3).
7. Write your README: the data, the model, the baseline comparison, a loss curve, and one thing that did not work. Push the folder to a new GitHub repository.

Sources

- PyTorch 2.15: a DataLoader over 10 rows with batch_size=4 gives batches of 4, 4 and 2, and 4 and 4 with drop_last=True; torch.as_tensor(X, dtype=torch.float32) converts float64 arrays. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), Python 3.12, November 2026.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B23-L1 Tensors, shape, dtype and shared memory

B23-L1 exercise 1

Hints. (1) Which side is Double? (2) What is NumPy's default float? (3) Convert the data to float32.

Answer. a) Build the tensor with `torch.as_tensor(X, dtype=torch.float32)` Because: r1) NumPy data is float64 and layer weights are float32. Converting the data once, at the edge, keeps the model in its default dtype.

`to_numpy()` gives float64, and `torch.tensor` keeps it. Layers hold float32 weights, so convert the data once with `torch.as_tensor(..., dtype=torch.float32)`.

Common wrong answers.

- `b`: That works, but doubles memory and slows every step. Convert the data instead.
- `c`: Rounding gives integers, int64, which still differ from float32 weights, and loses information.
- `d`: The device is not the problem. The dtypes differ: Double against Float.
- `a|r2`: Right fix, wrong reason. float32 is less precise than float64. It is the dtype layers use by default.

B23-L1 exercise 2

Hints. (1) Summing over a dimension removes it. (2) keepdim keeps it, with size 1. (3) unsqueeze(0) adds a dimension at the front.

Answer.

```
1 torch.Size([3]) torch.Size([4, 1]) torch.Size([1, 4, 3])
```

`sum(dim=0)` removes the row dimension, `keepdim=True` keeps the column dimension as size 1, and `unsqueeze(0)` adds a new first dimension.

Common wrong answers.

- `torch.Size([4]) torch.Size([4, 1]) torch.Size([1, 4, 3])`: dim=0 sums down the rows, removing the first dimension. Three columns remain.

- `torch.Size([3]) torch.Size([4]) torch.Size([1, 4, 3])`: keepdim=True keeps the summed dimension with size 1: (4, 1).

B23-L1 exercise 3

Hints. (1) Line the shapes up from the right: (3, 1) and (3,). (2) Both stretch to (3, 3). (3) Squares: 0, 1, 4 / 1, 0, 1 / 4, 1, 0.

Answer.

```
1 (3, 3) 1.3333
```

The predictions are perfect, yet the mean squared difference is 12 / 9 = 1.3333, because broadcasting compared every prediction with every target.

Common wrong answers.

- `(3,) 0.0`: (3, 1) minus (3,) broadcasts to (3, 3): every prediction against every target.
- `(3, 1) 0.0`: Shapes line up from the right. The trailing 1 stretches to 3, and the missing dimension stretches too.

B23-L1 exercise 4

Hints. (1) Which dtype do layer weights use? (2) Which method takes -1 to flatten? (3) `float32` and `reshape`.

Answer.

```
1 X_t = torch.as_tensor(X, dtype=torch.float32)
2 y_t = torch.as_tensor(y, dtype=torch.float32).reshape(-1)
```

`dtype=torch.float32` matches the weights, and `reshape(-1)` turns (n, 1) into (n,).

Common wrong answers.

- g1 = `float64`; g2 = `reshape`: Layer weights are float32. float64 data gives the Double and Float error.

B23-L1 exercise 5

Hints. (1) `flat = preds.reshape(-1)`. (2) Compare `flat.shape` with `y.shape`, and check `len(y) == 0`. (3) Wrap the mean in `float(...)`.

Answer.

```
1 import numpy as np
2
3
4 def mean_squared_error(preds: np.ndarray, y: np.ndarray) -> float:
5     """Mean squared error with predictions flattened to the target's
    shape.
```

```
6
7       Args:
8           preds: Predictions, shape (n,) or (n, 1).
9           y: Targets, shape (n,).
10
11      Returns:
12          The mean of the squared differences.
13
14      Raises:
15          ValueError: If the sizes differ or there are no rows.
16      """
17      flat = preds.reshape(-1)
18      if flat.shape != y.shape or len(y) == 0:
19          raise ValueError(f"need matching non-empty shapes, got
    {preds.shape} and {y.shape}")
20      return float(((flat - y) ** 2).mean())
```

Flattening first stops the (n, n) broadcast. The guards turn a shape mistake into a clear error instead of a plausible number.

B23-L1 exercise 6

Hints. (1) Does a slice copy the data? (2) Which operator changes an array in place? (3) Replace -= with a new array.

Answer. The bug is on line 8.

Fixed version:

```
1  import numpy as np
2
3
4  def centre_batches(X, batch_size):
5      batches = []
6      for start in range(0, len(X), batch_size):
7          batch = X[start:start + batch_size]
8          batch = batch - batch.mean(axis=0)
9          batches.append(batch)
10     return batches
```

`X[start:start + batch_size]` is a view of X, so `-=` writes the centred values back into X. `batch = batch - ...` builds a new array and leaves X alone.

Common wrong answers.

- Line 7: Slicing is fine on its own. It shares memory, so what happens next matters.
- Line 9: Appending stores the batch. The data changed one line earlier.

B23-L2 Autograd, gradients from the chain rule

B23-L2 exercise 1

Hints. (1) Should autograd record the weight update itself? (2) Which block turns recording off? (3) torch.no_grad().

Answer. a) Put the update inside `with torch.no_grad():` Because: r1) The update is not part of the model, so autograd must not record it. no_grad stops the recording and keeps w the same leaf tensor.

Inside `torch.no_grad()`, autograd records nothing, so the in-place update is allowed and `w` stays the leaf the optimiser and `backward()` work with.

Common wrong answers.

- `b`: That builds a new, non-leaf tensor. Its .grad stays None and the next step fails.
- `c`: Then autograd stops computing gradients for w, and training stops.
- `d`: The second call fails, because backward frees the graph, and gradients would add up anyway.
- `a|r2`: Right fix, wrong reason. It stops autograd recording the update.

B23-L2 exercise 2

Hints. (1) Differentiate w ** 2 + 2 * w. (2) Evaluate at w = 3. (3) Which tensor did you create directly?

Answer.

```
8.0 True False
```

dy/dw = 2w + 2 = 8 at w = 3. You created `w`, so it is a leaf. `y` is the result of operations.

Common wrong answers.

- `15.0 True False`: 15 is the value of y. The gradient is 2w + 2 = 8 at w = 3.
- `8.0 True True`: y comes from operations on w, so it is not a leaf.

B23-L2 exercise 3

Hints. (1) How far apart are w + h and w - h? (2) Which block stops autograd recording? (3) `2 * h` and `no_grad`.

Answer.

```
estimate = (f(w_plus) - f(w_minus)) / (2 * h)

with torch.no_grad():
    w -= lr * w.grad
```

The two points are `2 * h` apart. The update goes inside `torch.no_grad()`.

Common wrong answers.

- g1 = `h`; g2 = `no_grad`: w_plus and w_minus are 2h apart, so divide by 2 * h.

B23-L2 exercise 4

Hints. (1) `n, k = X.shape`. (2) Check `n == 0`, `len(y) != n` and `len(w) != k`. (3) `2 / n * X.T @ (X @ w - y)`.

Answer.

```python
import numpy as np


def mse_gradient(X: np.ndarray, y: np.ndarray, w: np.ndarray) -> np.ndarray:
    """Gradient of the mean squared error of a linear model.

    Args:
        X: Features, shape (n, k).
        y: Targets, shape (n,).
        w: Weights, shape (k,).

    Returns:
        The gradient, shape (k,).

    Raises:
        ValueError: If there are no rows or the shapes do not match.
    """
    n, k = X.shape
    if n == 0 or len(y) != n or len(w) != k:
        raise ValueError(f"shapes do not match: X {X.shape}, y {y.shape}, w {w.shape}")
    return 2 / n * X.T @ (X @ w - y)
```

The derivative of the squared error brings down a factor of 2, and the mean divides by n. The guards stop a silent broadcast from returning a wrong-shaped gradient.

B23-L2 exercise 5

Hints. (1) Start from `base = w.astype(float)`. (2) In the loop, copy `base` twice and move element `i` by `+h` and `-h`. (3) Divide the difference by `2 * h`.

Answer.

```python
from collections.abc import Callable

import numpy as np

```

```python

def numerical_gradient(
    f: Callable[[np.ndarray], float], w: np.ndarray, h: float = 1e-5
) -> np.ndarray:
    """Central-difference estimate of the gradient of f at w.

    Args:
        f: Function from a weight vector to a number.
        w: Point to differentiate at. Not changed.
        h: Step size.

    Returns:
        One estimate per element of w.
    """
    base = w.astype(float)
    grad = np.zeros_like(base)
    for i in range(len(base)):
        plus, minus = base.copy(), base.copy()
        plus[i] += h
        minus[i] -= h
        grad[i] = (f(plus) - f(minus)) / (2 * h)
    return grad
```

Float copies keep `w` safe and let `+h` work on integer input, where adding 1e-5 to an int array would truncate to 0. The two points are 2h apart.

B23-L2 exercise 6

Hints. (1) After the first update, is w still the tensor you created? (2) Does a non-leaf tensor get a .grad? (3) Update in place under no_grad, then reset the gradient.

Answer. The bug is on line 7.

Fixed version:

```python
import torch

w = torch.zeros(X.shape[1], requires_grad=True)
for step in range(100):
    loss = ((X @ w - y) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        w -= lr * w.grad
    w.grad.zero_()
```

`w = w - lr * w.grad` binds `w` to a new tensor built from operations, so it is no longer a leaf and its `.grad` stays `None`. Update in place inside `torch.no_grad()`, and reset `w.grad` after each step.

Common wrong answers.

- Line 6: backward() fills w.grad correctly on the first step. The next line replaces w.
- Line 3: requires_grad=True makes w a leaf, which is right. Something turns it into a non-leaf later.

B23-L3 Gradient descent, learning rates and optimisers

B23-L3 exercise 1

Hints. (1) What does a loss that grows each step mean? (2) Which features produce large gradients? (3) Smaller steps, scaled features.

Answer. a) Standardise the features and lower the learning rate Because: r1) A loss that grows each step is overshooting: steps too large for the gradients that unscaled features produce.

Rising loss then `nan` is divergence. Scaled features and a smaller rate keep each step from overshooting.

Common wrong answers.

- `b`: More steps of a diverging update only make the weights larger.
- `c`: The update rule is failing, not the model's capacity.
- `d`: A larger step overshoots further. That is how the loss reached nan.
- `a|r2`: Right fix, wrong reason. It is about step size against gradient size.

B23-L3 exercise 2

Hints. (1) One step multiplies p by 1 - 2 * lr. (2) The factors are 0.8, -0.8 and -1.2. (3) 4 * factor ** 5.

Answer.

```
0.1 1.3107
0.9 -1.3107
1.1 -9.9533
```

Factors 0.8, -0.8 and -1.2, applied five times to 4. A factor beyond -1 grows each step: divergence.

Common wrong answers.

- `0.1 1.3107 0.9 0.0 1.1 0.0`: Larger rates do not land on the minimum. Each step multiplies p by 1 - 2 * lr: -0.8 and -1.2.
- `0.1 1.3107 0.9 1.3107 1.1 9.9533`: 1 - 2 * lr is negative for both, and five steps is odd, so the sign flips.

B23-L3 exercise 3

Hints. (1) The optimiser with decoupled weight decay. (2) Which method applies the update? (3) `AdamW` and `step`.

Answer.

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

optimizer.zero_grad()
loss = loss_fn(model(x), y)
loss.backward()
optimizer.step()
```

`torch.optim.AdamW(model.parameters(), lr=1e-3)` builds the optimiser, and `optimizer.step()` applies the update after `backward()`.

Common wrong answers.

- g1 = `AdamW`; g2 = `zero_grad`: zero_grad clears the gradients backward() computed. step applies the update.

B23-L3 exercise 4

Hints. (1) The gradient of p ** 2 is 2p. (2) SGD subtracts lr * grad: p becomes 0.8p. (3) 4 * 0.8 ** 3.

Answer.

```
2.048
```

Each step is `p - 0.1 * 2p = 0.8p`. Three steps from 4 give 2.048.

Common wrong answers.

- `3.2`: That is after one step. The loop runs three: 4 * 0.8 ** 3.
- `1.6`: Each step multiplies p by 0.8, not subtracts 0.8.

B23-L3 exercise 5

Hints. (1) Check the arguments, then start with `w = np.zeros(X.shape[1])`. (2) In the loop: append the loss first, then update. (3) The update subtracts: `w - lr * 2 / len(y) * X.T @ (X @ w - y)`.

Answer.

```
import numpy as np


def gradient_descent(
    X: np.ndarray, y: np.ndarray, lr: float, steps: int
) -> tuple[np.ndarray, list[float]]:
    """Fit a linear model by full-batch gradient descent on mean squared
    error.

```

```
    Args:
        X: Standardised features, shape (n, k).
        y: Targets, shape (n,).
        lr: Learning rate.
        steps: Number of updates.

    Returns:
        The weights and the loss before each update.

    Raises:
        ValueError: If lr is not positive or steps is below 1.
    """
    if lr <= 0 or steps < 1:
        raise ValueError(f"need lr > 0 and steps >= 1, got {lr} and
{steps}")
    w = np.zeros(X.shape[1])
    losses: list[float] = []
    for _ in range(steps):
        losses.append(float(((X @ w - y) ** 2).mean()))
        w = w - lr * 2 / len(y) * X.T @ (X @ w - y)
    return w, losses
```

Recording the loss before each update makes the first value the starting loss, so the curve shows the whole run. The minus sign steps downhill.

B23-L3 exercise 6

Hints. (1) The gradient points uphill. (2) Which way should the weights move? (3) Subtract the step.

Answer. The bug is on line 10.

Fixed version:

```
import numpy as np


def gradient_descent(X, y, lr, steps):
    w = np.zeros(X.shape[1])
    losses = []
    for _ in range(steps):
        losses.append(float(((X @ w - y) ** 2).mean()))
        grad = 2 / len(y) * X.T @ (X @ w - y)
        w = w - lr * grad
    return w, losses
```

The gradient points in the direction the loss increases. Adding it climbs the loss surface, whatever the rate. Subtract it.

Common wrong answers.

- Line 9: The gradient formula is right. What happens to it on the next line?
- Line 8: Recording the loss is right. That is how you saw it rise.

B23-L4 Models as nn.Module, layers, activations and parameters

B23-L4 exercise 1

Hints. (1) What is a product of two matrices? (2) What does ReLU do to negative values? (3) Linear after linear is linear.

Answer. a) Two linear layers in a row equal one linear layer. It needs an activation such as nn.ReLU() between them Because: r1) x @ W1.T @ W2.T is x times one combined matrix, so depth adds nothing without a non-linear step.

Composed linear maps are one linear map. A non-linear activation between layers lets the model represent curves and interactions.

Common wrong answers.

- `b`: Any width collapses to one linear map without an activation.
- `c`: Sequential registers both layers. They both train, but together they are still linear.
- `d`: The output size matches the target: one value for one outcome.
- `a|r2`: Right fix, wrong reason. ReLU adds non-linearity, which lets depth help.

B23-L4 exercise 2

Hints. (1) Each Linear(a, b) holds a * b weights and b biases. (2) ReLU has no parameters. (3) Weight shape is (out_features, in_features).

Answer.

```
193 torch.Size([2, 3])
```

176 + 17 = 193 parameters. The weight is (out, in) because the layer computes `x @ W.T + b`.

Common wrong answers.

- `176 torch.Size([2, 3])`: Biases count too: 10 * 16 + 16 and 16 * 1 + 1.
- `193 torch.Size([3, 2])`: Linear stores its weight as (out, in), so Linear(3, 2) holds a (2, 3) weight.

B23-L4 exercise 3

Hints. (1) The parent class sets up registration first. (2) A list type that nn.Module tracks. (3) `__init__` and `ModuleList`.

Answer.

```python
class Stack(nn.Module):
    def __init__(self, sizes: list[int]) -> None:
        super().__init__()
        self.layers = nn.ModuleList(nn.Linear(a, b) for a, b in
    zip(sizes, sizes[1:]))
```

`super().__init__()` prepares registration, and `nn.ModuleList` registers each layer it holds.

Common wrong answers.

- g1 = `__init__`; g2 = `list`: nn.list does not exist, and a plain list hides the layers from parameters(). Use nn.ModuleList.

B23-L4 exercise 4

Hints. (1) Weights are (out, in), so multiply by their transpose: `X @ W1.T`. (2) Check `X.shape[1] == W1.shape[1]` and `W2.shape[1] == W1.shape[0]`, and both bias lengths. (3) `np.maximum(X @ W1.T + b1, 0) @ W2.T + b2`.

Answer.

```python
import numpy as np


def mlp_forward(
    X: np.ndarray, W1: np.ndarray, b1: np.ndarray, W2: np.ndarray, b2:
    np.ndarray
) -> np.ndarray:
    """Forward pass of Linear, ReLU, Linear with PyTorch's (out, in)
    weights.

    Args:
        X: Inputs, shape (n, in).
        W1: First weight, shape (hidden, in).
        b1: First bias, shape (hidden,).
        W2: Second weight, shape (out, hidden).
        b2: Second bias, shape (out,).

    Returns:
        Outputs, shape (n, out).

    Raises:
        ValueError: If the shapes do not fit together.
    """
    if X.shape[1] != W1.shape[1] or W2.shape[1] != W1.shape[0]:
        raise ValueError(f"weights do not fit: X {X.shape}, W1
    {W1.shape}, W2 {W2.shape}")
    if len(b1) != W1.shape[0] or len(b2) != W2.shape[0]:
        raise ValueError(f"biases do not fit: b1 {b1.shape}, b2
    {b2.shape}")
    return np.maximum(X @ W1.T + b1, 0) @ W2.T + b2
```

Each Linear layer is `x @ W.T + b` with an (out, in) weight. ReLU sits between them. Checking shapes first turns a broadcasting accident into a clear error.

B23-L4 exercise 5

Hints. (1) `zip(sizes, sizes[1:])` gives consecutive pairs. (2) Each pair adds `a * b + b`. (3) Check `len(sizes) < 2 or min(sizes) < 1` first.

Answer.

```python
def count_linear_params(sizes: list[int]) -> int:
    """Parameters in Linear layers between consecutive sizes.

    Args:
        sizes: Layer widths from input to output.

    Returns:
        Weights plus biases over all layers.

    Raises:
        ValueError: If fewer than two sizes are given, or any is below 1.
    """
    if len(sizes) < 2 or min(sizes) < 1:
        raise ValueError(f"need at least two sizes of 1 or more, got {sizes}")
    return sum(a * b + b for a, b in zip(sizes, sizes[1:]))
```

Each layer has one weight per input-output pair and one bias per output. The count for [10, 16, 1] matches `sum(p.numel() for p in model.parameters())`: 193.

B23-L4 exercise 6

Hints. (1) Where does parameters() look for layers? (2) Is a Python list a module? (3) nn.ModuleList.

Answer. The bug is on line 8.

Fixed version:

```python
import torch
from torch import nn


class Stack(nn.Module):
    def __init__(self, sizes):
        super().__init__()
        self.layers = nn.ModuleList(nn.Linear(a, b) for a, b in zip(sizes, sizes[1:]))

    def forward(self, x):
        for layer in self.layers[:-1]:
            x = torch.relu(layer(x))
```

```
13          return self.layers[-1](x)
```

`parameters()` walks registered submodules. A plain list is not one, so the model reports no parameters. `nn.ModuleList` registers each layer.

Common wrong answers.

- Line 7: super().**init**() is right and in the right place. The layers are stored somewhere parameters() cannot see.
- Line 12: The forward pass runs. The layers it uses were never registered.

B23-L5 Loss functions, logits and target shapes

B23-L5 exercise 1

Hints. (1) Which loss name contains Logits? (2) Where does the sigmoid happen? (3) Raw logits in, float targets.

Answer. a) `nn.Linear(hidden, 1)` with no activation, and `nn.BCEWithLogitsLoss()` on float targets Because: r1) BCEWithLogitsLoss applies the sigmoid itself, stably, so the model outputs raw logits.

The model outputs one logit. `BCEWithLogitsLoss` applies the sigmoid inside the loss, stably, against float targets.

Common wrong answers.

- `b`: That applies the sigmoid twice: once in the model and once in the loss.
- `c`: MSE on probabilities gives weak gradients for confident wrong answers. Binary targets want a binary loss.
- `d`: CrossEntropyLoss expects one logit per class and integer class indices.
- `a|r2`: Right choice, wrong reason. It is about the loss applying the sigmoid itself.

B23-L5 exercise 2

Hints. (1) What shape do (3, 1) and (3,) broadcast to? (2) Mean of 0, 1, 4, 1, 0, 1, 4, 1, 0. (3) After squeeze(1), the shapes match.

Answer.

```
1 1.3333 0.0
```

The first call compares a 3 by 3 table and averages 12 / 9. Squeezing gives matching shapes and the correct 0.0.

Common wrong answers.

- `0.0 0.0`: Without squeeze, (3, 1) against (3,) broadcasts to (3, 3). Every prediction is compared with every target.

- `1.3333 1.3333`: squeeze(1) gives (3,), the target's shape, so the perfect predictions score 0.0.

B23-L5 exercise 3

Hints. (1) pos_weight scales the positive rows. Which ratio makes them count more? (2) Which method removes a dimension of size 1? (3) `n_neg / n_pos` and `squeeze`.

Answer.

```
loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([n_neg / n_pos]))
loss = loss_fn(model(x).squeeze(1), y.float())
```

`pos_weight = n_neg / n_pos` balances the classes, and `squeeze(1)` turns (n, 1) logits into (n,).

Common wrong answers.

- g1 = `n_pos / n_neg`; g2 = `squeeze`: That shrinks the weight of the rare class. pos_weight is negatives over positives.

B23-L5 exercise 4

Hints. (1) Check `logits.shape != targets.shape or logits.size == 0` first. (2) Use `np.maximum(z, 0)`, `z * y` and `np.log1p(np.exp(-np.abs(z)))`. (3) Wrap the mean in `float(...)`.

Answer.

```
import numpy as np


def bce_with_logits(logits: np.ndarray, targets: np.ndarray) -> float:
    """Binary cross-entropy on logits, in the numerically stable form.

    Args:
        logits: Raw scores, shape (n,).
        targets: 0 or 1, shape (n,).

    Returns:
        The mean loss.

    Raises:
        ValueError: If the shapes differ or there are no rows.
    """
    if logits.shape != targets.shape or logits.size == 0:
        raise ValueError(f"need matching non-empty shapes, got {logits.shape} and {targets.shape}")
    z, y = logits, targets
    return float(np.mean(np.maximum(z, 0) - z * y + np.log1p(np.exp(-np.abs(z)))))
```

`exp(-abs(z))` is at most 1, so it never overflows, and `log1p` stays accurate when that value is tiny. The result equals the textbook formula wherever both are finite.

B23-L5 exercise 5

Hints. (1) Validate with `np.isin(y_train, [0, 1]).all()` and `y_train.size`. (2) `n_pos = int(y_train.sum())`. (3) Return `(len(y_train) - n_pos) / n_pos`.

Answer.

```python
import numpy as np


def positive_weight(y_train: np.ndarray) -> float:
    """pos_weight for BCEWithLogitsLoss: negatives / positives.

    Args:
        y_train: Training targets of 0 and 1.

    Returns:
        The ratio of negative to positive rows.

    Raises:
        ValueError: If there are no rows, no positives, or values other
    than 0 and 1.
    """
    if y_train.size == 0 or not np.isin(y_train, [0, 1]).all():
        raise ValueError("targets must be a non-empty array of 0 and 1")
    n_pos = int(y_train.sum())
    if n_pos == 0:
        raise ValueError("no positive rows in the training data")
    return (len(y_train) - n_pos) / n_pos
```

With 6 negatives and 2 positives, each positive row counts 3 times, so the two classes carry equal total weight. The checks stop a 2 in the targets from inflating the count.

B23-L5 exercise 6

Hints. (1) What does the loss do to its input first? (2) What does the model's last layer do? (3) One sigmoid too many.

Answer. The bug is on line 7.

Fixed version:

```python
from torch import nn


class Recommender(nn.Module):
    def __init__(self, n_features):
        super().__init__()
```

```
7         self.net = nn.Sequential(nn.Linear(n_features, 16), nn.ReLU(),
      nn.Linear(16, 1))
8
9     def forward(self, x):
10        return self.net(x).squeeze(1)
11
12
13 loss_fn = nn.BCEWithLogitsLoss()
```

The model applies a sigmoid, and the loss applies another. Outputs of sigmoid lie in (0, 1), so the second sigmoid maps them into (0.5, 0.73), and probabilities never drop below 0.5. Remove `nn.Sigmoid()` and apply `torch.sigmoid` only when reporting probabilities.

Common wrong answers.

- Line 13: BCEWithLogitsLoss is the right loss. It expects logits, and the model does not give it logits.
- Line 10: squeeze(1) matches the target's shape. That is right.

B23-L6 Dataset, DataLoader and preprocessing fitted on training rows

B23-L6 exercise 1

Hints. (1) Does the order of rows change a mean? (2) What does a new order give the gradient steps? (3) Shuffle where it changes training.

Answer. a) Training benefits from a new row order each epoch. Validation loss does not depend on order, so shuffling it adds nothing Because: r1) Varied batches give each epoch different gradient steps. A validation metric averages over all rows, whatever their order.

Shuffling varies the batches, and so the steps, during training. A validation loss averages over every row, so its order does not matter.

Common wrong answers.

- `b`: Order does not reveal targets. Shuffling validation is pointless, not harmful.
- `c`: A mean over all validation rows is the same in any order.
- `d`: Each DataLoader has its own shuffle setting.
- `a|r2`: Right answer, wrong reason. Shuffling does not speed anything up. It varies the gradient steps.

B23-L6 exercise 2

Hints. (1) 10 rows in batches of 4. (2) What happens to the 2 rows left over? (3) drop_last removes the short batch.

Answer.

```
[4, 4, 2]
[4, 4]
```

Ten rows give batches of 4, 4 and 2. `drop_last=True` drops the batch of 2.

Common wrong answers.

- `[4, 4] [4, 4]`: Without drop_last, the last two rows form a short final batch.
- `[4, 4, 2] [4, 4, 2]`: drop_last=True drops the final batch when it is shorter than batch_size.

B23-L6 exercise 3

Hints. (1) One method answers len(dataset). (2) The other answers dataset[i]. (3) `__len__` and `__getitem__`.

Answer.

```python
class SurveyRows(Dataset):
    def __init__(self, X, y):
        self.X = torch.as_tensor(X, dtype=torch.float32)
        self.y = torch.as_tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, i):
        return self.X[i], self.y[i]
```

`__len__` gives the row count and `__getitem__` returns row `i`. The DataLoader uses both to build batches.

Common wrong answers.

- g1 = `__len__`; g2 = `__iter__`: DataLoader asks for rows by index. That is **getitem**.

B23-L6 exercise 4

Hints. (1) Check for no rows and NaN first. (2) `mean = X_train.mean(axis=0)`, `std = X_train.std(axis=0)`, then `np.where(std == 0, 1.0, std)`. (3) Scale both arrays with the same mean and std.

Answer.

```python
import numpy as np
```

```python


def standardise(X_train: np.ndarray, X_other: np.ndarray) ->
    tuple[np.ndarray, np.ndarray]:
    """Scale features with statistics from the training rows only.

    Args:
        X_train: Training features, shape (n, k).
        X_other: Validation or test features, shape (m, k).

    Returns:
        Both arrays, scaled with the training mean and standard deviation.

    Raises:
        ValueError: If X_train has no rows, or either array holds NaN.
    """
    if len(X_train) == 0 or np.isnan(X_train).any() or
    np.isnan(X_other).any():
        raise ValueError("impute missing values first, and pass at least
    one training row")
    mean = X_train.mean(axis=0)
    std = X_train.std(axis=0)
    std = np.where(std == 0, 1.0, std)
    return (X_train - mean) / std, (X_other - mean) / std
```

Training statistics only, applied to both arrays, keep validation rows out of the preprocessing. The zero guard keeps a constant column finite.

B23-L6 exercise 5

Hints. (1) Check both arguments first. (2) `order = rng.permutation(n)`, once. (3) `[order[i : i + batch_size] for i in range(0, n, batch_size)]`.

Answer.

```python
import numpy as np


def batch_indices(n: int, batch_size: int, rng: np.random.Generator) ->
    list[np.ndarray]:
    """Shuffled row indices split into batches, as a shuffling DataLoader
    does.

    Args:
        n: Number of rows.
        batch_size: Rows per batch. The last batch may be shorter.
        rng: Seeded generator.

    Returns:
        Index arrays that cover every row once.

    Raises:
        ValueError: If n or batch_size is below 1.
```

```
    """
    if n < 1 or batch_size < 1:
        raise ValueError(f"need n >= 1 and batch_size >= 1, got {n} and {batch_size}")
    order = rng.permutation(n)
    return [order[i : i + batch_size] for i in range(0, n, batch_size)]
```

One permutation, split in order, covers every row exactly once per epoch, and the seed fixes the order.

B23-L6 exercise 6

Hints. (1) Which rows does X.mean(axis=0) use? (2) Has the split happened yet? (3) Split first, then fit the scaling on training rows.

Answer. The bug is on line 5.

Fixed version:

```
import numpy as np


def prepare(X, y, rng, n_train):
    order = rng.permutation(len(X))
    train, val = order[:n_train], order[n_train:]
    mean, std = X[train].mean(axis=0), X[train].std(axis=0)
    std = np.where(std == 0, 1.0, std)
    return (X[train] - mean) / std, y[train], (X[val] - mean) / std, y[val]
```

The mean and standard deviation come from every row, including validation, before the split. Split first, compute the statistics on the training rows, and apply them to both sets.

Common wrong answers.

- Line 6: A seeded permutation is a good way to split. The problem happens before the split.
- Line 8: Returning the split rows is right. They were scaled with the wrong statistics.